



A deep dive into one of the most critical algorithms in computer science — powering every dependency manager, build system, and task scheduler in the modern world.

Topic Classification

Module Identity

Topic Name: Topological Sort

Category: Graph Algorithms

Domain: Computer Science / DAG
Algorithms / Ordering Problems

Difficulty: Advanced

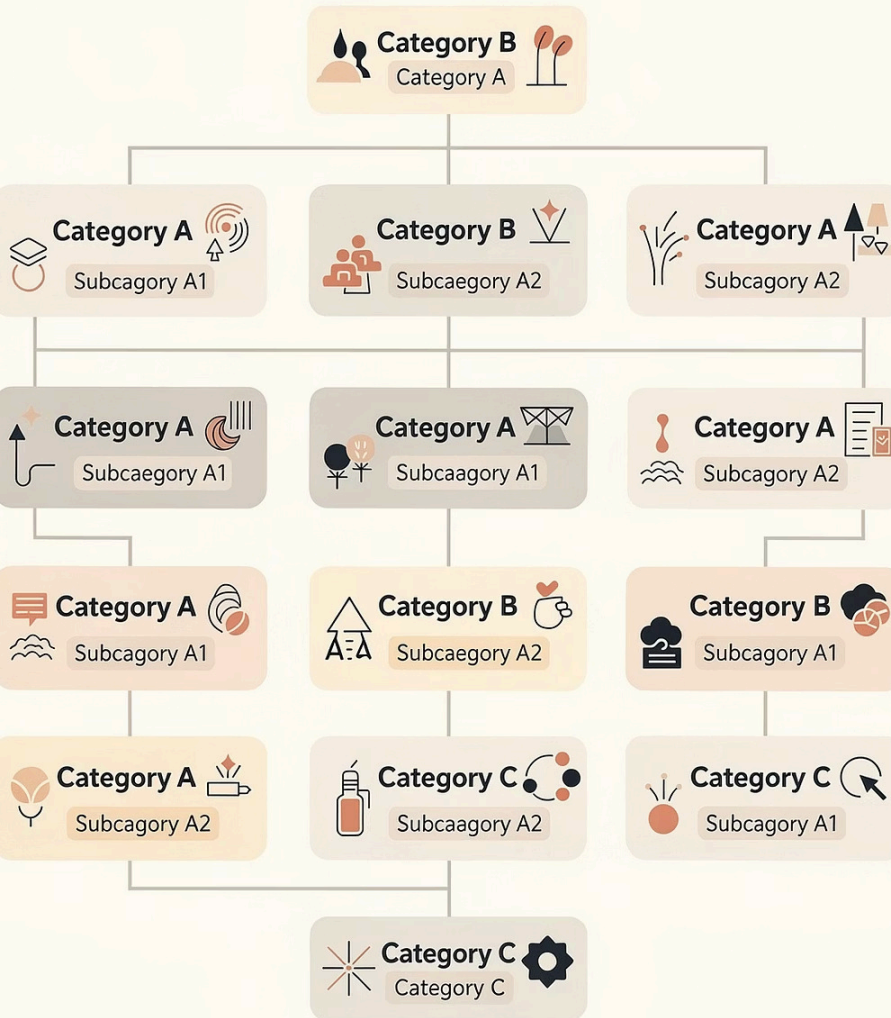
Prerequisites

- Graph Basics
- BFS (Kahn's) & DFS (DFS-based)
- Directed Graphs
- Cycle Detection

Industry Relevance

Critical — Dependency resolution (npm, pip, Maven), build systems (Make, Bazel, Gradle), task scheduling, course prerequisites, spreadsheet evaluation, compiler symbol resolution.

Recommended Learning Path



What Is Topological Sort?

Formal Definition

A **topological ordering** of a directed acyclic graph $G = (V, E)$ is a linear ordering of all vertices such that for every directed edge $(u \rightarrow v)$, vertex **u appears before v** in the ordering.

A topological ordering exists **if and only if the graph is a DAG** (no directed cycles). If a cycle exists, no valid ordering is possible.

 Topological sort is not a sort in the comparison sense — it produces **one valid ordering among potentially many** that respects all dependency constraints.

Core Invariant

For every edge $(u \rightarrow v)$ in the graph, **u must appear at an earlier position than v** in the output. This invariant, when satisfied for ALL edges simultaneously, constitutes a valid topological ordering.

Multiple Valid Orderings — Example DAG

DAG: $A \rightarrow C \rightarrow E$
 $B \rightarrow C$
 $B \rightarrow D \rightarrow E$

Valid topological orderings:

$[A, B, C, D, E]$ ← A before B, both before C

$[B, A, C, D, E]$ ← B before A (also valid)

$[B, D, A, C, E]$ ← D before C **is** fine

$[A, B, D, C, E]$ ← also valid

All are correct — ANY valid ordering **is** accepted.

 The problem asks for ANY valid ordering — not a unique one. Multiple correct answers always exist for non-trivial DAGs.

Why Does It Exist?

Many real-world problems require ordering items where some must come before others, and these constraints form a directed graph.

Problem	Vertices	Edges (u→v means)	What Topo Sort Gives
Package install	Packages	u must install before v	Valid install order
Course enrolment	Courses	u is prerequisite for v	Valid enrolment order
Build system	Files/targets	u must compile before v	Valid build order
Task scheduling	Tasks	u must complete before v	Valid execution order
Spreadsheet cells	Cells	cell u feeds into cell v	Evaluation order
Database migrations	Migrations	migration u must run before v	Valid migration order

What Happens Without Topological Sort

● Circular Errors

Circular dependency errors that are hard to trace if cycle detection is not done first

● Build Failures

Non-deterministic build failures from incorrect ordering

● Runtime Crashes

Runtime failures when a module is loaded before its dependencies

● Deadlocks

Deadlocks in task scheduling systems

⚠ The npm circular dependency error, Maven build failure, and Kubernetes pod startup ordering issues all trace back to topological ordering problems.

Evolution & History

Topological sort has been a cornerstone of computer science for over six decades, evolving from academic theory to the backbone of modern software infrastructure.



How It Works — The Algorithms

Algorithm 1: Kahn's Algorithm (BFS-Based) — Production Standard

Intuition: Repeatedly remove vertices with no incoming edges (no unsatisfied dependencies), appending them to the result. When a vertex is removed, it satisfies dependencies for its neighbours — decrement their in-degree. A neighbour whose in-degree hits 0 becomes the next "ready" vertex.

```
FUNCTION kahn_topological_sort(graph, V):
  // Step 1: Compute in-degree for every vertex
  in_degree = [0] * V
  FOR u in range(V):
    FOR v in graph[u]:
      in_degree[v] += 1

  // Step 2: Initialize queue with all sources
  queue = deque()
  FOR u in range(V):
    IF in_degree[u] == 0:
      queue.append(u)

  // Step 3: Process queue
  result = []
  WHILE queue not empty:
    u = queue.popleft()
    result.append(u)
    FOR v in graph[u]:
      in_degree[v] -= 1
      IF in_degree[v] == 0:
        queue.append(v)

  // Step 4: Cycle detection
  IF len(result) < V:
    RETURN ERROR "Cycle detected"
  RETURN result
// Time: O(V+E) Space: O(V)
```

Algorithm 2: DFS-Based Topological Sort

Intuition: In DFS, a vertex finishes (all its descendants are processed) before we move on. Since u must appear BEFORE its dependents, and finishes AFTER them, the finish order reversed gives topological order.

```
FUNCTION dfs_topological_sort(graph, V):
  visited = [FALSE] * V
  result = []

  FUNCTION dfs(u):
    visited[u] = TRUE
    FOR v in graph[u]:
      IF NOT visited[v]:
        dfs(v)
    result.append(u) // AFTER all descendants

  FOR u in range(V):
    IF NOT visited[u]:
      dfs(u)
  RETURN result[::-1] // Reverse finish order
// Time: O(V+E) Space: O(V)
```

⚠ This version does NOT detect cycles automatically. Add 3-state colouring: 0=unvisited, 1=in_progress, 2=done. If we visit a vertex in state 1 → CYCLE.

Kahn's vs. DFS — Detailed Comparison

Dimension	Kahn's (BFS)	DFS-Based
Algorithm basis	BFS with in-degree tracking	DFS with finish-time ordering
Cycle detection	✅ Free — result.size < V	⚠ Requires 3-state colouring
Ordering flexibility	Priority queue → lexicographically smallest	Stack-based → depends on DFS order
Iterative?	✅ Naturally iterative	❌ Recursive by default
Stack overflow risk	❌ None	✅ Possible on deep DAGs
Production preference	✅ Preferred (Kahn's)	Good for DFS-heavy codebases

☒ **Production rule:** Use Kahn's algorithm by default. It provides cycle detection for free, is naturally iterative (no stack overflow risk), and is easier to extend for lexicographic topo sort.

Lexicographically Smallest Topological Order

Replace the plain queue in Kahn's with a **min-heap (priority queue)**. This gives the lexicographically smallest valid topological ordering.

```
FUNCTION lex_smallest_topo(graph, V):
  in_degree = compute_in_degrees(graph, V)
  heap = min_heap()
  FOR u in range(V):
    IF in_degree[u] == 0:
      heap.push(u)
  result = []
  WHILE heap not empty:
    u = heap.pop() // Always process smallest available
    result.append(u)
    FOR v in graph[u]:
      in_degree[v] -= 1
      IF in_degree[v] == 0:
        heap.push(v)
  IF len(result) < V: RETURN ERROR "Cycle"
  RETURN result
// Time: O((V+E) log V) — log factor from heap
// Use case: deterministic build ordering, reproducible CI/CD
```

DP on DAG — The Most Powerful Topo Sort Application

Once a topological order is computed, **dynamic programming on the DAG** solves longest/shortest path, counting paths, and more in O(V+E) — much faster than Dijkstra or Bellman-Ford on general graphs.

Longest Path in a DAG (Critical Path)

```
FUNCTION longest_path_dag(graph, V, weights):
  topo_order = kahn_topological_sort(graph, V)
  dist = [-INFINITY] * V
  FOR u in topo_order:
    IF dist[u] == -INFINITY:
      dist[u] = 0 // Source vertex
  FOR u in topo_order:
    IF dist[u] == -INFINITY: CONTINUE
    FOR (v, w) in graph[u]:
      IF dist[u] + w > dist[v]:
        dist[v] = dist[u] + w
  RETURN MAX(dist)
// Application: project management CPM/PERT
```

All Topological Orderings (Backtracking)

```
FUNCTION all_topological_sorts(graph, V):
  in_degree = compute_in_degrees(graph, V)
  visited = [FALSE] * V
  result = []; all_results = []

  FUNCTION backtrack():
    found_any = FALSE
    FOR u in range(V):
      IF NOT visited[u] AND in_degree[u] == 0:
        visited[u] = TRUE; result.append(u)
        FOR v in graph[u]: in_degree[v] -= 1
        backtrack()
        FOR v in graph[u]: in_degree[v] += 1
        result.pop(); visited[u] = FALSE
    found_any = TRUE
    IF NOT found_any AND len(result) == V:
      all_results.append(copy(result))
    backtrack()
  RETURN all_results
// Exponential in worst case — use only for small graphs
```

Walkthrough — Course Schedule

Courses: 0(Math), 1(Physics), 2(CS101), 3(Algorithms)

Prerequisites: 0→1, 0→2, 1→3, 2→3

in_degree: [0, 1, 1, 2]

Initial queue: [0] ← only Math has no prereq

Step 1: Process 0 (Math)

result = [0]

Neighbours: 1,2 → in_degree=[0,0,0,2]

Queue: [1, 2]

Step 2: Process 1 (Physics)

result = [0, 1]

Neighbours: 3 → in_degree[3]=1

Queue: [2]

Step 3: Process 2 (CS101)

result = [0, 1, 2]

Neighbours: 3 → in_degree[3]=0 → enqueue

Queue: [3]

Step 4: Process 3 (Algorithms)

result = [0, 1, 2, 3]

len(result)=4=V → no cycle ✓

Valid order: Math→Physics→CS101→Algorithms

🕒 Cycle detection is **FREE** in Kahn's: if result.size < V, some vertices were stuck in a cycle and never reached in-degree 0.

DFS with Cycle Detection (3-State)

```
FUNCTION dfs_topo_with_cycle_check(graph, V):
  state = [0] * V // 0=unvisited,1=in_progress,2=done
  result = []
  cycle = [FALSE]

  FUNCTION dfs(u):
    IF cycle[0]: RETURN
    state[u] = 1 // Mark as in progress
    FOR v in graph[u]:
      IF state[v] == 1: // Back edge → cycle
        cycle[0] = TRUE; RETURN
      IF state[v] == 0:
        dfs(v)
    state[u] = 2 // Mark as done
    result.append(u)

  FOR u in range(V):
    IF state[u] == 0:
      dfs(u)
  IF cycle[0]:
    RETURN ERROR "Cycle detected"
  RETURN result[::-1]
```

Types & Variants

Variant	Algorithm	Extra Cost	Use Case
Any valid order	Kahn's or DFS	$O(V+E)$	General dependency resolution
Lexicographically smallest	Kahn's + min-heap	$O((V+E) \log V)$	Reproducible builds
With cycle detection	Kahn's (free) or 3-state DFS	$O(V+E)$	Package manager validation
All valid orderings	Backtracking	Exponential	Testing, analysis
Parallel topo sort	Kahn's per-level	$O(V+E)$	Parallel task execution
Online (dynamic edges)	Pearce-Kelly algorithm	$O(V+E)$ amortised	Live-updating dependency graphs

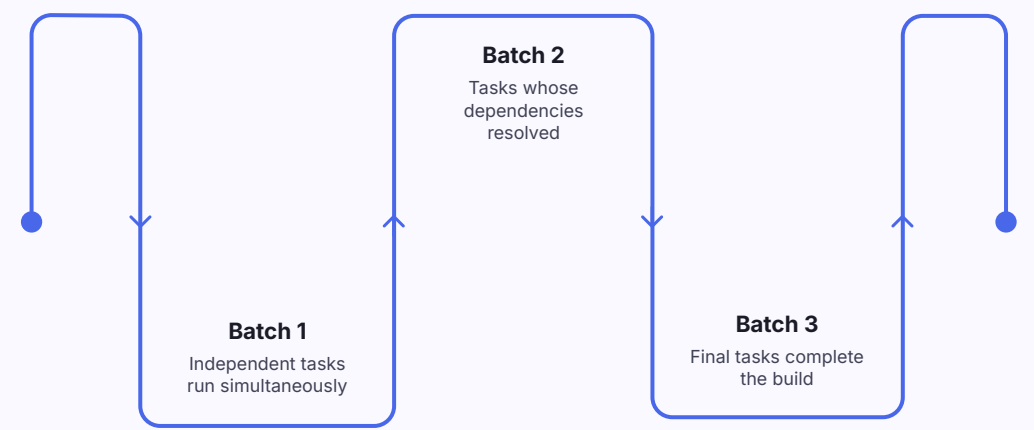
Parallel Topological Sort — Executing Independent Tasks Simultaneously

```
FUNCTION parallel_topo_sort(graph, V):
  in_degree = compute_in_degrees(graph, V)
  current_batch = [u for u if in_degree[u] == 0]
  all_batches = []

  WHILE current_batch:
    all_batches.append(current_batch)
    next_batch = []
    FOR u in current_batch:
      FOR v in graph[u]:
        in_degree[v] -= 1
        IF in_degree[v] == 0:
          next_batch.append(v)
    current_batch = next_batch

  RETURN all_batches
// Each batch = one "time step" of parallel execution
// len(all_batches) = minimum time with infinite workers
```

Application: Minimum number of semesters to complete courses (LeetCode #1136). Each semester = one batch; students take all available courses simultaneously.



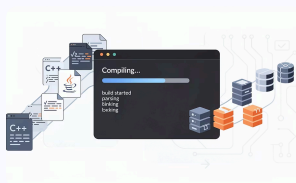
Real-World Use Cases



npm / pip / Maven Dependency Resolution

Every package manager in the world uses topological sort. When installing package A (which depends on B and C, which both depend on D), Kahn's algorithm determines the correct install order and detects circular dependencies.

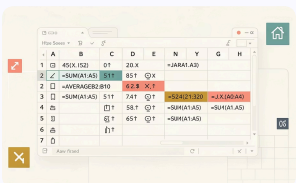
Dependency graph:
 $A \rightarrow B \rightarrow D$
 $A \rightarrow C \rightarrow D; C \rightarrow E$
Kahn's result: [D, E, B, C, A]
Circular dep: $\text{result.size} < V$
 $\rightarrow$ "ERESOLVE: Circular dependency"



GNU Make / Bazel Build System

Build systems use parallel topological sort to compile only what's necessary, in the right order, with maximum parallelism. Bazel's internal build graph has millions of nodes at Google scale.

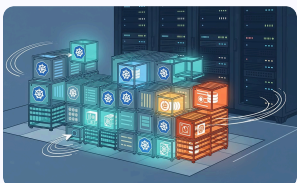
Parallel batches:
Batch 1: [main.c, util.c, util.h]
 $\leftarrow$ compile
simultaneously
Batch 2: [main.o, util.o]
 $\leftarrow$ both depend on
Batch 1
Batch 3: [main]
 $\leftarrow$ depends on Batch 2



Spreadsheet Formula Evaluation

Excel and Google Sheets build a dependency graph of cell references and evaluate in topological order. Circular references (A1 = B1; B1 = A1) are detected as cycles.

Dependency graph:
 $A1, A2 \rightarrow B1 \rightarrow C1 \rightarrow D1$
 $B1 \longrightarrow$
D1
Topo **order**: [A1, A2, B1, C1, D1]
Circular **ref** $\rightarrow$ "Circular reference error"



Kubernetes Service Startup Ordering

In microservices, services have startup dependencies. Docker Compose and Helm use topological sort to determine which services start first, and which can start in parallel.

Topo sort: [database, cache, api, frontend]
Parallel batches:
Batch 1: [database, cache]
Batch 2: [api]
Batch 3: [frontend]

Implementations & Examples

Implementation 1: Complete Kahn's with Diagnostics

```
FUNCTION kahn_with_diagnostics(graph, V, vertex_names=None):
    in_degree = [0] * V
    FOR u in range(V):
        FOR v in graph[u]:
            in_degree[v] += 1

    queue = deque([u for u in range(V) if in_degree[u] == 0])
    result = []; steps = 0

    WHILE queue:
        batch = list(queue) // All currently processable vertices
        PRINT f"Step {steps}: Can process {[vertex_names[u] for u in batch]}"
        u = queue.popleft()
        result.append(u)
        FOR v in graph[u]:
            in_degree[v] -= 1
            IF in_degree[v] == 0:
                queue.append(v)
        steps += 1

    IF len(result) < V:
        stuck = [u for u in range(V) if u NOT in result]
        RETURN ERROR f"Cycle involves vertices: {stuck}"
    RETURN result
```

Implementation 2: Minimum Semesters (Parallel Topo Sort)

```
FUNCTION min_semesters(n, relations):
    graph = [[] for _ in range(n+1)]
    in_degree = [0] * (n+1)
    FOR (u, v) in relations:
        graph[u].append(v)
        in_degree[v] += 1

    queue = deque([u for u in range(1, n+1)
                    if in_degree[u] == 0])
    semesters = 0; taken = 0

    WHILE queue:
        semesters += 1
        FOR _ in range(len(queue)): // Entire batch = 1 semester
            u = queue.popleft()
            taken += 1
            FOR v in graph[u]:
                in_degree[v] -= 1
                IF in_degree[v] == 0:
                    queue.append(v)

    IF taken < n: RETURN -1 // Cycle detected
    RETURN semesters

// Key: snapshot len(queue) before processing batch
```

Implementation 3: Alien Dictionary

```
FUNCTION alien_order(words):
    // Step 1: Collect all unique characters
    graph = {c: set() for word in words for c in word}
    in_degree = {c: 0 for word in words for c in word}

    // Step 2: Extract ordering from adjacent word pairs
    FOR i in range(len(words) - 1):
        w1 = words[i]; w2 = words[i+1]
        min_len = MIN(len(w1), len(w2))
        IF len(w1) > len(w2) AND w1[:min_len] == w2[:min_len]:
            RETURN "" // Invalid: longer word comes first
        FOR j in range(min_len):
            IF w1[j] != w2[j]:
                IF w2[j] NOT in graph[w1[j]]:
                    graph[w1[j]].add(w2[j])
                    in_degree[w2[j]] += 1
        BREAK // Only first difference matters

    // Step 3: Kahn's topo sort on character graph
    queue = deque([c for c in in_degree if in_degree[c] == 0])
    result = []
    WHILE queue:
        c = queue.popleft(); result.append(c)
        FOR next_c in graph[c]:
            in_degree[next_c] -= 1
            IF in_degree[next_c] == 0:
                queue.append(next_c)
    IF len(result) < len(in_degree): RETURN "" // Cycle
    RETURN "".join(result)

// Time: O(C + N×L) where C=chars, N=words, L=word length
```

Hands-On Labs

Lab 1: Build a Task Scheduler (Kahn's Algorithm)

Context: You are building a CI/CD pipeline runner. Each task has dependencies.

Tasks and dependencies:

```
lint    → [test]
unit_test → [test]
test    → [build]
build   → [docker_build]
docker_build → [deploy]
deploy  → []
```

1. Build the directed graph (each dependency = directed edge)
2. Run Kahn's algorithm — print each step showing which tasks become "ready"
3. Compute parallel batches (tasks that can run simultaneously)
4. Add an artificial cycle: `deploy → lint`. Re-run and observe cycle detection output

✓ Expected parallel batches (no cycle):

Batch 1: [lint, unit_test] ← no dependencies
Batch 2: [test] ← depends on both
Batch 3: [build]
Batch 4: [docker_build]
Batch 5: [deploy]

Lab 2: DP on DAG — Course Credits Maximisation

Extended problem: Each course has a credit value. Maximise total credits taking all courses in valid prerequisite order.

```
// dp[v] = max credits reachable ending at v
FUNCTION max_credits(graph, weights, V):
    topo = kahn_topological_sort(graph, V)
    dp = [w for w in weights]
    FOR u in topo:
        FOR v in graph[u]:
            IF dp[u] + weights[v] > dp[v]:
                dp[v] = dp[u] + weights[v]
    RETURN MAX(dp)
```

Test Case

Courses: 0(Math,3cr)→1(Physics,4cr)→3(Quantum,5cr)
0(Math,3cr)→2(CS,3cr)→3(Quantum,5cr)

Topo order: [0, 1, 2, 3]

$dp[0] = 3$

$dp[1] = dp[0] + 4 = 7$

$dp[2] = dp[0] + 3 = 6$

$dp[3] = \max(dp[1]+5, dp[2]+5) = \max(12, 11) = 12$

Max credits = 12 (path: Math→Physics→Quantum)

Performance Considerations

Algorithm	Time	Space	Notes
Kahn's Topo Sort	$O(V+E)$	$O(V)$	Optimal; cycle detection free
DFS Topo Sort	$O(V+E)$	$O(V)$	Same; cycle detection requires 3-state
Lex-Smallest Kahn's	$O((V+E) \log V)$	$O(V)$	Min-heap instead of queue
All Orderings	$O(V! \times (V+E))$	$O(V)$	Exponential; small graphs only
Parallel Topo Sort	$O(V+E)$	$O(V)$	Returns batches; $O(\text{depth})$ wall-clock
DP on DAG (after topo)	$O(V+E)$	$O(V)$	After topo order computed
Longest Path (DAG)	$O(V+E)$	$O(V)$	Only polynomial because it's a DAG

Why Longest Path on DAG is Polynomial but NP-Hard Generally

General Graph

Longest path = **NP-Hard** — no polynomial algorithm known.
The problem is equivalent to Hamiltonian path.

DAG

Longest path = **$O(V+E)$** via DP in topological order. Each vertex processed exactly once. No revisiting possible (no cycles).

 **Why DAGs are computationally valuable:** Many NP-hard problems on general graphs become polynomial on DAGs.

- Longest path: NP-hard $\rightarrow O(V+E)$ on DAG
- Counting paths: #P-hard $\rightarrow O(V+E)$ on DAG
- Reachability: $O(V \times E) \rightarrow O(V+E)$ on DAG with topo preprocessing

Topological structure is the key difference.

Design & Architectural Considerations

Handling Cycles Gracefully in Production

In production dependency systems, cycles must be detected and clearly reported with the offending cycle path:

```
FUNCTION kahn_with_cycle_path(graph, V):
  in_degree = compute_in_degrees(graph, V)
  queue = deque([u for u if in_degree[u] == 0])
  result = []

  WHILE queue:
    u = queue.popleft()
    result.append(u)
    FOR v in graph[u]:
      in_degree[v] -= 1
      IF in_degree[v] == 0:
        queue.append(v)

  IF len(result) == V:
    RETURN result, None

  // Find and report the cycle
  cycle_vertices = [u for u in range(V)
                    if u NOT in set(result)]
  cycle_path = find_cycle_in_subgraph(graph, cycle_vertices)
  RETURN None, f"Cycle: {cycle_path}"

// npm, pip, Maven all report the cycle path:
// "npm ERR! ERESOLVE: Cycle: pkg-a → pkg-b → pkg-a"
```

Dynamic Topological Sort (Online Updates)

Standard Kahn's and DFS-based algorithms are static — they require the full graph upfront. For systems where edges are added dynamically, use the **Pearce-Kelly algorithm**:

Add Edge (u→v)

If v already comes before u in current order, find and fix the affected region

Time Complexity

$O((V+E) \log V)$ amortised across all insertions

Used In

Incremental build systems (iBAT, Pants build), live dependency graph editors

Topological Sort in Distributed Systems

For very large graphs (millions of nodes in a build graph at Google/Facebook), topological sort is parallelised:

Distributed Kahn's:

Round 0: All workers find vertices with $\text{in_degree}=0 \rightarrow$ add to frontier

Each round: Distribute frontier vertices across workers

Each worker: Processes its vertices, sends "in_degree decrements" to neighbours' owning workers

Terminate: When no new vertices added
If $\text{processed_count} < V \rightarrow$ cycle exists

Used in: Google Blaze (internal Bazel), Facebook Buck build systems

Best Practices

✓ Do's

Default Choice

Use Kahn's algorithm — cycle detection is free and it's naturally iterative

Cycle Reporting

Always report WHICH vertices are in the cycle, not just "cycle detected"

Parallel Execution

Use parallel topo sort (level-by-level Kahn's) to find tasks that can run concurrently

DP on DAG

After topo sort, DP over vertices in topological order for $O(V+E)$ path/count problems

Testing

Test: empty graph, single vertex, disconnected DAG, graph with exactly one cycle, complete DAG

Edge Validation

Validate for cycles when accepting user-defined dependency graphs (build configs, API schemas)

✗ Don'ts

Algorithm

Don't use DFS topo sort without 3-state cycle detection — silent wrong results on cyclic input

Assumption

Don't assume a dependency graph is acyclic — always run cycle detection

Performance

Don't apply Dijkstra for shortest path when graph is a DAG — topo DP is $O(V+E)$ vs $O((V+E) \log V)$

Reporting

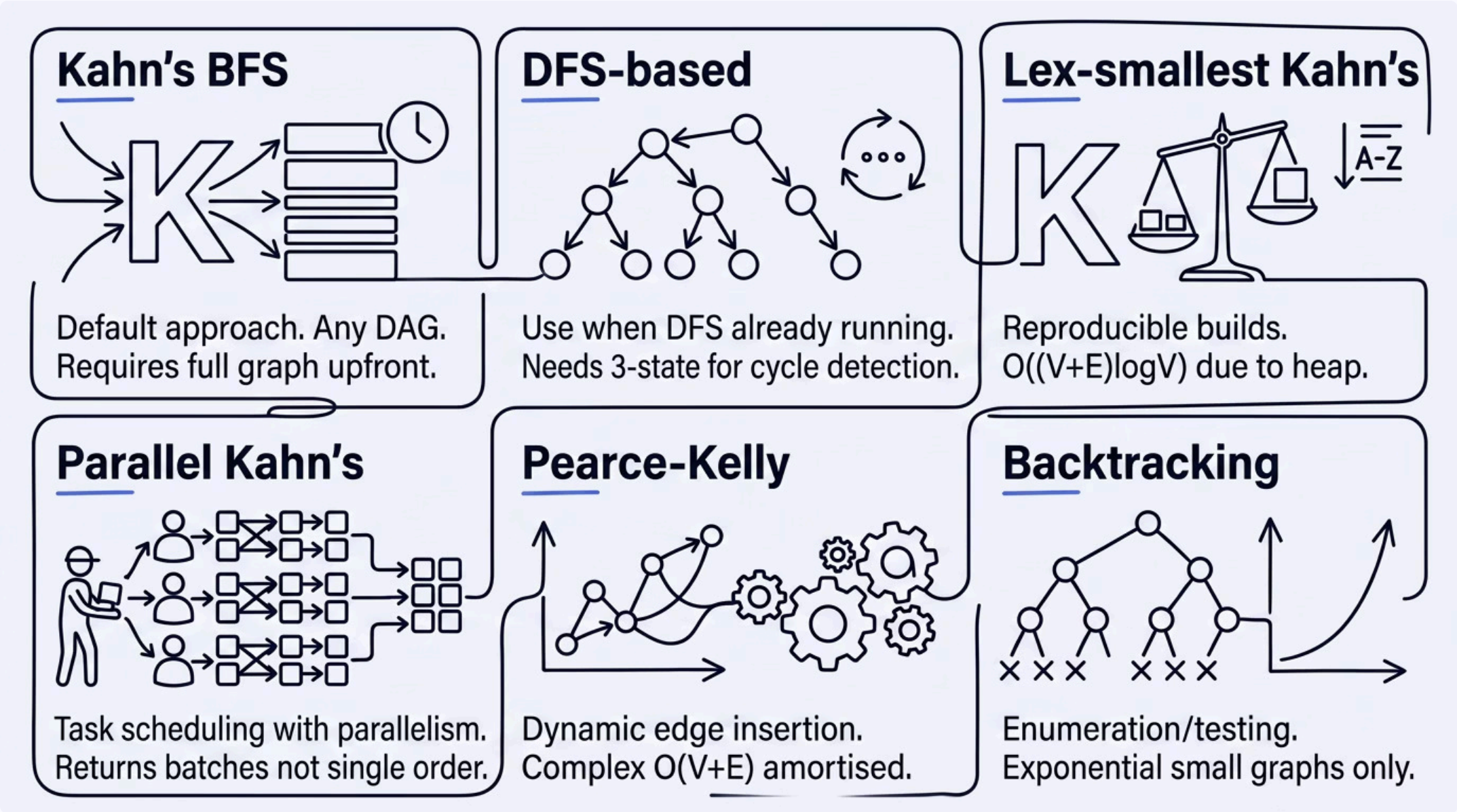
Don't just return "error: cycle" without identifying which vertices form the cycle

Common Mistakes & Pitfalls

Level	Mistake	Impact	Fix
Intermediate	DFS topo sort without cycle detection	Silently returns wrong order when cycle exists	Add 3-state colouring; or use Kahn's with free cycle detection
Intermediate	Forgetting to decrement ALL neighbours in Kahn's	Some vertices never reach in_degree=0	Ensure <code>FOR v in graph[u]: in_degree[v] -= 1</code> processes ALL outgoing edges of u
Intermediate	Using topological sort on an undirected graph	Meaningless result	Topo sort is ONLY defined for directed acyclic graphs
Advanced	Applying Dijkstra on DAG when negative edges exist	Wrong shortest paths	DAG DP handles negative edges correctly; Dijkstra requires non-negative weights
Advanced	Parallel topo sort reporting wrong batch count	Incorrect minimum semesters/rounds	Snapshot <code>len(queue)</code> BEFORE processing the batch; process exactly that many nodes per round
Advanced	Not using modular arithmetic for counting paths	Integer overflow	Path counts grow exponentially; always <code>paths[v] = (paths[v] + paths[u]) % MOD</code>

❌ The most dangerous mistake is using DFS-based topo sort without 3-state cycle detection. It will silently produce an incorrect ordering on cyclic graphs — no error, no warning, just wrong results.

Comparison with Alternatives



Approach	When to Use	Limitation
Kahn's (BFS)	Default; any DAG ordering need	Requires full graph upfront
DFS-based	When DFS already running for other purposes	Needs 3-state for cycle detection
Lex-smallest Kahn's	Reproducible/deterministic build order	$O((V+E) \log V)$ due to heap
Parallel Kahn's	Task scheduling with parallelism	Returns batches, not single order
Pearce-Kelly (online)	Dynamic edge insertion	Complex; $O(V+E)$ amortised
Backtracking all-orders	Enumeration/testing	Exponential; small graphs only

Learning Summary & Revision

Key Takeaways

1 The Core Definition

Topological sort orders DAG vertices such that every edge ($u \rightarrow v$) has u before v . Exists **if and only if the graph is a DAG** — a cycle means no valid ordering is possible.

2 Kahn's Algorithm

Use in-degree tracking; process 0-in-degree vertices via queue; cycle detection is free (`result.size < V`). Naturally iterative — the production default.

3 DFS-Based

Vertices appended in reverse finish order. Requires 3-state colouring for cycle detection. Good when DFS is already being run for other purposes.

4 Parallel & DP Extensions

Parallel topo sort (level-by-level Kahn's) minimises wall-clock time. DP on DAG after topo sort solves longest/shortest path and path counting in $O(V+E)$.

5 The DAG Advantage

Longest path on DAG = $O(V+E)$; on general graphs = NP-hard. Topological structure is the key difference that unlocks polynomial solutions.

Revision Cheat Sheet

TOPOLOGICAL SORT QUICK REFERENCE	
DEFINITION: Linear order where every edge ($u \rightarrow v$) has u before v	
EXISTS: If and only if graph is a DAG (no directed cycles)	
KAHN'S ALGORITHM (BFS-based):	
1. Compute <code>in_degree[]</code> for all vertices	
2. Queue all vertices with <code>in_degree == 0</code>	
3. Dequeue $u \rightarrow$ append to result $\rightarrow$ decrement neighbours' <code>in_deg</code> $\rightarrow$ enqueue neighbours with <code>in_degree</code> now <code>== 0</code>	
4. if <code>len(result) < V</code> $\rightarrow$ CYCLE DETECTED	
Time: $O(V+E)$ Cycle detection: FREE	
DFS-BASED:	
Append vertex AFTER all its descendants are processed	
Reverse the result list $\rightarrow$ topological order	
Cycle detection: 3 -state (0 =unvisited/ 1 =in_progress/ 2 =done)	
PARALLEL TOPO SORT:	
Process entire queue per round = one batch runs concurrently	
Number of rounds = minimum time with unlimited parallelism	
DP ON DAG (after topo sort):	
Longest path: <code>dp[v] = max(dp[u] + w(u,v))</code> for all $u \rightarrow v$	
Count paths: <code>dp[v] = Σ dp[u]</code> for all $u \rightarrow v$	
Shortest path: <code>dp[v] = min(dp[u] + w(u,v))</code> for all $u \rightarrow v$	
All: $O(V+E)$ — process in topo order , no revisiting	
KEY DISTINCTION:	
Kahn's: free cycle detection, iterative, production default	
DFS: needs extra cycle check, recursive, depth-first order	

Memory Aids



"Kahn's = in-degree queue"

Zero in-degree means ready; drain the queue



"DFS = finish late, appear first"

Last to finish goes first (reversed)



"Cycle = result.size < V"

Some vertices stuck waiting forever



"DAG DP = process once in order"

Topo order guarantees all predecessors done before current



"Parallel = snapshot queue size"

That many tasks run in the current round

Common Interview Questions

- What is topological sort? Under what condition does it exist?
- Explain Kahn's algorithm. How does it detect cycles for free?
- Explain DFS-based topological sort. Why do we reverse the finish order?
- Compare Kahn's and DFS-based — when would you prefer each?
- How would you find the lexicographically smallest topological ordering?
- How does parallel topological sort work? What does each "round" represent?
- How do you find the longest path in a DAG? Why can't this be done on general graphs efficiently?
- Given a list of courses with prerequisites, find the minimum number of semesters to finish all.
- How does npm detect circular dependencies? What algorithm does it use?
- Explain DP on DAG for counting paths. Why does it require topological order?

Recommended Next Topics

- 

Shortest Path Algorithms

Dijkstra, Bellman-Ford, DAG DP
- 

Minimum Spanning Tree

Prim's, Kruskal's on undirected weighted graphs
- 

Strongly Connected Components

Tarjan's, Kosaraju's; condensation DAG
- 

2-SAT Problem

Solved via SCC on implication graph
- 

Network Flow

Ford-Fulkerson, Edmonds-Karp
- 

DP on Trees

Generalisation of DAG DP to tree structures



End of Module

Graph Patterns — Advanced Module 2

Topological Sort

You have completed the full Topological Sort module — from formal definition and algorithm internals, through real-world applications in npm, Make, Kubernetes, and spreadsheets, to advanced DP on DAG techniques and production best practices.

Next Up

Shortest Path on DAG → Strongly
Connected Components

Core Skill Unlocked

Dependency resolution, parallel
scheduling, and DAG dynamic
programming

Production Takeaway

Default to Kahn's algorithm — free cycle detection, iterative, and battle-tested
at scale